Cloud Computing (CS3081): Mini Project Final Report

Knowledge Distillation and Microservice Deployment on a Local Kubernetes Cloud

Repository: https://github.com/KANAND6031/Cloud-Computing-Mini-Project | **Evaluated Git Tag:** `final`

1 Task 1: Object Storage and Parallel Teacher Labelling

1.1 Architecture and Setup

To simulate an enterprise S3 cloud environment on a local development laptop, we initialized a lightweight multi-node Kubernetes cluster using `k3d` configured in `cluster/k3d.yaml`. The virtual topology contains one control-plane server node (`server-0`), two agent worker nodes (`agent-0`, `agent-1`), and an integrated server load balancer (`serverlb`) mapping host port 8080 to the internal Traefik ingress proxy.

Within the cluster, we deployed SeaweedFS into a dedicated namespace (`data`) with a PersistentVolumeClaim. SeaweedFS serves standard Amazon S3 REST APIs on internal cluster port 8333 (forwarded to port 9000 for host-side tooling). We created two isolated S3 buckets: `datasets` (storing raw command pools, shard splits, and cleaned datasets) and `models` (storing distilled model checkpoints). The teacher model, `Phi-3.5-mini-instruct` (3.8 billion parameters), was hosted using an Ollama daemon. Due to intense CPU saturation during 3,000-sample sequence generation, we ran Ollama natively on the laptop host on port 11434 rather than inside a k3d container, preventing worker node OOM crashes while pointing cluster workers to the host network.

1.2 Design Decisions and Engineering Rationale

- **Deterministic Line-Modulo Sharding:** Rather than using random sampling or non-deterministic file splits, we partitioned `data/commands_pool.jsonl` using index modulo sharding: line index i is assigned to worker k if $i \pmod N = k$, where $N = 3$ concurrent workers. This guaranteed mathematical completeness: exactly 1,000 commands per shard, zero unassigned commands, and zero cross-worker duplicate IDs.
- **Strict Schema Enforcement in Prompt:** In `prompts/teacher_prompt.txt`, we constrained the teacher to emit exclusively a four-key JSON object (`intent`, `date`, `time`, `place`) with temperature 0 and seed 42. Slots not mentioned in the prompt were forced to literal `null`.
- **Hierarchical Cleaning Pipeline:** In `worker/clean.py`, checks were applied sequentially: JSON syntax → exact key verification → valid 8-intent taxonomy → hallucination detection (verifying that non-null slot strings literally exist in the prompt) → deduplication (retaining the command with the lowest numeric ID).

1.3 Measured Empirical Results

The entire 3,000-command dataset was processed across 3 shards and cleaned. Table 1 lists the exact measured counts.

Category	Rejection Reason / Verification Criterion	Measured Count / Percentage
Total Raw Input	Initial lines extracted from `commands_pool.jsonl`	3,000 (100.00%)
Rule (a) Rejected	Malformed or unparseable JSON syntax	0 (0.00%)
Rule (b) Rejected	Keys not exactly {`intent`, `date`, `time`, `place`}	0 (0.00%)
Rule (c) Rejected	Intent not in the approved 8-class domain taxonomy	0 (0.00%)
Rule (d) Rejected	Hallucinated slots (non-null slot text absent from raw command)	37 (1.23%)
Rule (e) Deduplicated	Identical command strings (kept lowest ID)	202 (6.73%)
Retained Clean Records	Written to `datasets/v1/train.jsonl` in ID order	**2,761 (92.03%)**
Dataset Checksum	`c8cf810cd1d4ee304c5b04ee629a3523dcd8c211674201abd3749d72a9fdead6`	

Table 1: Task 1 dataset sharding, cleaning, and cryptographic verification counts.

1.4 Key Assumptions and Modeling Boundaries

- *Assumption 1 (Dataset Formatting):* We assumed all input commands in `commands_pool.jsonl` were UTF-8 encoded, single-line JSON records.
- *Assumption 2 (Uniform Generation Time):* We assumed command string complexity was evenly distributed across line numbers, producing roughly balanced execution times across shards.

1.5 Identified Limitations and Proposed Improvements

- **Limitation (Straggler Sensitivity):** Static modulo sharding cannot handle variable sentence complexity. If one shard contains longer commands, that worker lags behind, leaving other workers idle.
- **Improvement:** Replace static sharding with a dynamic queue (e.g., Redis List or RabbitMQ) where worker pods fetch individual command tasks as workers become free.

2 Task 2: Student Training (Sequence-Level Distillation)

2.1 Architecture and Setup

For model distillation, we fine-tuned `EleutherAI/pythia-70m` (70.4 million parameters) on CPU using PyTorch and HuggingFace Transformers. Per assignment rules, training was restricted strictly to the first 1,500 records of our cleaned dataset (`datasets/v1/train.jsonl`). All hyperparameters, library dependencies, and seeds were recorded in `training/config.yaml`. Training runs were executed via a single Makefile command: `make train` (and verified on a small slice using `make train SUBSET=50`).

Each training instance was formatted as an autoregressive text generation problem:

```
Command: <user command string>
JSON: {"intent": "<intent>", "date": ..., "time": ..., "place": ...}
```

2.2 Design Decisions and Engineering Rationale

- **Sequence-Level Distillation over Logit-Matching:** We trained the student using teacher-generated sequence outputs as supervised targets. Logit-based distillation (minimizing KL-divergence between output probability distributions) is fundamentally impossible in this project because `Phi-3.5` uses a 32,064-token vocabulary, whereas `Pythia-70M` uses a 50,304-token vocabulary. Because token vocabularies and tokenizers do not align, logit-level matching cannot be computed.
- **CPU Convergence Strategy:** To prevent memory spikes on our laptop, we trained with batch size 8, learning rate 5×10^{-5} using AdamW, linear learning rate decay, and max sequence length 128 tokens.
- **Multi-Version Checkpoints:** We trained two distinct versions: Version 1 (seed 42, 3 epochs) and Version 2 (seed 43, 4 epochs). Both were uploaded to SeaweedFS S3 (`s3://models/models/v1/` and `v2/`). Version 1 was published as a GitHub Release asset (`v1.0.0`).

2.3 Measured Empirical Results

The training loss recorded in `training/loss.csv` demonstrated steady convergence:

- **Version 1 Loss Curve:** Epoch 1: **0.6974** → Epoch 2: **0.1652** → Epoch 3: **0.0807**.
- **Version 2 Loss Curve:** Epoch 1: 0.6511 → Epoch 2: 0.1482 → Epoch 3: 0.0763 → Epoch 4: **0.0412**.
- **Release Checksum (v1):** `dbc2a82188fff0df1ca8c8ab67cbfb9f5c79258e48036861978438260b80bae6`.

We evaluated all models against the 100 gold records in `data/validation.jsonl` using `tools/score.py`. As shown in Table 2, the untrained baseline failed to generate valid JSON (0.0%), while our distilled student reached **98.0% JSON validity**.

Model Evaluated	JSON Validity	Intent Acc.	Date Acc.	Time Acc.	Place Acc.	Full F
Untrained Pythia-70M	0.0%	0.0%	0.0%	0.0%	0.0%	0.0
Teacher (`Phi-3.5-mini`)	100.0%	68.0%	65.0%	78.0%	82.0%	45.
Distilled Student (Raw Output)	92.0%	54.0%	48.0%	67.0%	74.0%	9.0
Distilled Student (Schema-Normalized)	**98.0%**	**54.0%**	**50.0%**	**71.0%**	**78.0%**	**11.**

Table 2: Task 2 empirical model evaluation metrics on `data/validation.jsonl` (100 samples).

2.4 Key Assumptions and Modeling Boundaries

- *Assumption 1 (Teacher Optimality):* Sequence distillation assumes the teacher's greedily decoded text represents near-optimal ground truth for the command domain.
- *Assumption 2 (Isolated Single-Turn Prompts):* Commands are assumed to be independent, single-turn interactions with no cross-command state.

2.5 Identified Limitations and Proposed Improvements

- **Limitation (Capacity Ceiling):** With only 70M parameters, the student exhibits low full-frame accuracy (11.0% vs 45.0% teacher), struggling on complex sentences containing multiple concurrent slot entities.
- **Improvement:** Apply Low-Rank Adaptation (LoRA) fine-tuning on a 0.5B model (such as `Qwen2.5-0.5B`) or employ constrained beam search decoding during inference.

3 Task 3: Inference Service in Container

3.1 Architecture and Setup

We packaged the trained student model into a production HTTP microservice in `service/app.py` using FastAPI and Uvicorn. The service exposes three endpoints:

- `POST /parse`: Accepts {`"text": "..."`}, runs PyTorch inference, validates the output against `data/schema.json`, and injects tracking headers `X-Model-Version` and `X-Served-By`.
- `GET /healthz`: Liveness probe returning `HTTP 200 OK` as long as the web worker process is running.
- `GET /readyz`: Readiness probe returning `HTTP 503 Service Unavailable` while model weights are downloading from S3, switching to `HTTP 200 OK` once the model weights are resident in RAM.

3.2 Design Decisions and Engineering Rationale

- **Zero Model Weights in Container Image:** To adhere to cloud-native 12-factor principles, model weights were not baked into the Docker image. At startup, the container reads `MODEL_URI`, `MODEL_VERSION`, and `S3_ENDPOINT`, dynamically streaming the model archive from SeaweedFS via `boto3` using credentials securely mounted from a Kubernetes Secret.
- **Multi-Stage CPU-Only Docker Build:** Default PyTorch wheels bundle large CUDA/NVIDIA shared libraries exceeding 4 GB. We built a multi-stage Docker image on `python:3.11-slim` using `--extra-index-url https://download.pytorch.org/whl/cpu`, eliminating GPU libraries.
- **Defensive Output Validation and Fault Injection:** The service validates JSON output against schema keys. When `FAULT_INJECT=1` is passed via environment variables, the service deliberately replaces the output with `"not json"`, returning `HTTP 500` {`"error": "invalid_model_output"`}, proving that broken responses never return HTTP 200.

3.3 Measured Empirical Results

- **Container Image Size:** Total virtual image size is **1.63 GB** (uncompressed filesystem content is only **351 MB**), well below the 2 GB assignment ceiling.
- **Stress Test under Cgroups (1 CPU core, 1 GiB memory):** We sent 50 sequential requests against the resource-constrained container using `docker run --cpus=1.0 --memory=1024m`:
  - Completed Requests: **50 / 50 (100.0% success, 0 OOM kills)**.
  - Peak Memory Observed: **348.10 MiB** (leaving 679.9 MiB or 66.4% headroom below the 1 GiB limit).
  - Measured Latency Distribution: Median (p50) = **2,904.73 ms**, p95 = **3,598.11 ms**, Minimum = **1,411.02 ms**.

3.4 Key Assumptions and Modeling Boundaries

- *Assumption 1 (Memory Determinism):* We assumed PyTorch CPU inference memory remains stable across consecutive generation loops without garbage accumulation.
- *Assumption 2 (Storage Proximity):* We assumed S3 object storage is reachable over local cluster networks with negligible latency during pod boot.

3.5 Identified Limitations and Proposed Improvements

- **Limitation (Synchronous CPU Inference):** PyTorch autoregressive token generation runs synchronously, monopolizing the CPU core. Under concurrent traffic, this blocks Uvicorn's asyncio event loop, causing health probe timeouts.
- **Improvement:** Export the model to ONNX format or run inference inside an asynchronous C++ engine (such as Triton Inference Server or vLLM) with continuous request batching.

4 Task 4: Kubernetes Deployment, Scaling, and Reliability

4.1 Architecture and Setup

We deployed the containerized inference service into the `app` namespace across two worker nodes. The Kubernetes stack comprises:

- **Workload Deployment:** 2 initial replicas of `inference-service` with resource requests (`250m` CPU, `512Mi` RAM) and hard limits (`1000m` CPU, `1024Mi` RAM). Configured with `initialDelaySeconds: 5` and `periodSeconds: 5` on `/readyz` and `/healthz`, alongside a `preStop: sleep 5` lifecycle hook.
- **Service and Ingress Routing:** A `ClusterIP` Service exposing port 8000 connected to a Traefik Ingress route mapping `/parse` to host port 8080.
- **Horizontal Pod Autoscaler (HPA):** Configured in `deployment/hpa.yaml` with min replicas 1, max replicas 2, and target CPU utilization of 70%.

4.2 Design Decisions and Engineering Rationale

- **Graceful Shutdown via `preStop`:** During rolling updates or pod deletions, Traefik's endpoint routing table requires several seconds to propagate pod removals. By introducing `preStop: sleep 5`, we allowed inflight connections to complete before the pod received `SIGKILL`, achieving zero dropped requests.
- **Readiness-Gated Traffic Routing:** Because the service downloads model weights at pod boot, routing traffic immediately would cause 502/503 errors. The readiness probe prevents endpoints from receiving traffic until memory initialization is complete.

4.3 Measured Empirical Results

- **Traffic Load Balancing:** During a 104-request benchmark through Traefik, traffic was distributed with exact parity:
  - Replica `agent-0`: **52 requests (50.0%)**
  - Replica `agent-1`: **52 requests (50.0%)**
- **HPA Autoscaling Dynamics:**
  - Scaling Up ($1 \rightarrow 2$ pods): Triggered under stress load, completed in **20.3 seconds**.
  - Scaling Down ($2 \rightarrow 1$ pod): Stabilized and scaled down after load ceased in **129.1 seconds** (respecting cooldown).
- **Zero-Downtime Rolling Update and Rollback:** We updated `MODEL_VERSION` and `MODEL_URI` from v1 to v2 during continuous traffic, verified headers, and rolled back via `kubectl rollout undo`:
  - Total Requests: **104** | Failed Requests: **0** | Dropped Request Rate: **0.00%**.
  - Active header transition: `v1` (52 req) $\rightarrow$ `v2` (52 req) $\rightarrow$ `v1` post-rollback.
- **Fault Recovery under Load:** Deleting an active pod during load resulted in Kubernetes restoring the replica count within **17.8 seconds** with **0 failed requests (0.00% error rate)**.
- **Pod Cold-Start Latency (S3 Weight Download to Ready):**
  - Run 1: 6.34s | Run 2: 7.13s | Run 3: 8.01s $\implies$ **Median Cold Start: 7.13 seconds**.

4.4 Key Assumptions and Modeling Boundaries

- *Assumption 1 (Metrics Server Sampling):* Kubernetes Metrics Server polls cgroup stats every 15 seconds, introducing an inherent 15–30 second latency in HPA reactivity.
- *Assumption 2 (Node Homogeneity):* Both k3d agent nodes share host CPU and RAM evenly without hardware virtualization throttling.

4.5 Identified Limitations and Proposed Improvements

- **Limitation (Single-Host Co-Location):** Because all nodes run on a single laptop kernel, node failure cannot be simulated authentically; a kernel crash kills the entire cluster.
- **Improvement:** Deploy across multi-zone managed Kubernetes (e.g. EKS/GKE) with cluster-autoscaler and topology spread constraints.

5 Task 5: Subnetting and Network Access Control

5.1 Architecture and Setup

We partitioned the private class-B network address space `10.20.0.0/16` into four functional subnets following Variable Length Subnet Masking (VLSM). The subnets were allocated sequentially from largest to smallest requirement, minimizing IP fragmentation:

1. `app`: Application compute tier housing microservice pods.
2. `public`: DMZ subnet housing public-facing ingress load balancers.
3. `data`: Secure storage tier housing SeaweedFS S3 object storage.
4. `management`: Administrative bastion subnet for administrative access.

5.2 Design Decisions and Engineering Rationale

- **Prefix Size Optimization:** Subnet sizes were computed via $2^{32-\text{prefix}} - 2 \geq$ Hosts Needed. We selected the smallest standard CIDR prefix satisfying each constraint (Table 3).
- **Stateless NACL Rule Semantics:** Network Access Control Lists (NACLs) are strictly stateless; unlike stateful security groups, they maintain no connection state tables. Consequently, return traffic for established TCP connections must be explicitly permitted in both directions.
- **Rule Ordering Precedence:** NACL engines evaluate rules in numerical order (lowest rule number first) with first-match termination. To effectively mitigate abusive clients, deny rules must strictly precede permissive accept rules.

5.3 Measured Empirical Results

The VLSM network allocation calculations are detailed in Table 3.

Subnet	Hosts Req.	Network CIDR	Subnet Mask	Usable Host Range	Broadcast Address	Usable
`app`	1,000	`10.20.0.0/22`	`255.255.252.0`	`10.20.0.1` – `10.20.3.254`	`10.20.3.255`	1,02
`public`	250	`10.20.4.0/24`	`255.255.255.0`	`10.20.4.1` – `10.20.4.254`	`10.20.4.255`	25
`data`	100	`10.20.5.0/25`	`255.255.255.128`	`10.20.5.1` – `10.20.5.126`	`10.20.5.127`	12
`management`	20	`10.20.5.128/27`	`255.255.255.224`	`10.20.5.129` – `10.20.5.158`	`10.20.5.159`	30

Table 3: Task 5 Variable Length Subnet Masking (VLSM) allocation table for `10.20.0.0/16`.

Our NACL ruleset for the public subnet (`network/nacl.json`) was verified using `tools/nacl_sim.py`:

- **Inbound Rules:**
  - Rule 50: `DENY` source `198.51.100.0/24` on all ports (blocks known abusive range before any allows).
  - Rule 100: `ALLOW` TCP 80 from `0.0.0.0/0` (HTTP public traffic).
  - Rule 110: `ALLOW` TCP 443 from `0.0.0.0/0` (HTTPS public traffic).
  - Rule 120: `ALLOW` TCP 22 from `10.20.5.128/27` (SSH exclusively from management).
  - Rule 130: `ALLOW` TCP ports 1024–65535 from `10.20.0.0/22:8000` (return responses from app tier).
  - Default Rule (*): `DENY` all other inbound packets.
- **Outbound Rules:**
  - Rule 100: `ALLOW` TCP port 8000 to `10.20.0.0/22` (forwarding proxy requests to app tier).
  - Rule 110: `ALLOW` TCP ports 1024–65535 to `0.0.0.0/0` (return responses to external clients).
  - Default Rule (*): `DENY` all other outbound packets.
- **Packet Simulation Verification:** Passed **6 / 6 sample packets (100%)** and **21 / 21 edge-case packets (100%)**.

5.4 Key Assumptions and Modeling Boundaries

- *Assumption 1 (Ephemeral Port Range):* Client operating systems use IANA standard ephemeral ports 1024–65535 for TCP return traffic.
- *Assumption 2 (Direct Intra-VPC Routing):* Traffic between subnets is routed directly within the VPC without intermediary NAT gateways.

5.5 Identified Limitations and Proposed Improvements

- **Limitation (SSH Port Exposure):** Opening port 22 directly from the management subnet increases attack surface if an admin workstation is compromised.
- **Improvement:** Eliminate SSH entirely in favor of AWS Systems Manager (SSM) Session Manager or Kubernetes Ephemeral Debug Containers with identity-aware RBAC.

6 Task 6: Reproducibility, Testing, and CI/CD Pipeline

6.1 Architecture and Setup

To ensure deterministic reproduction, the entire platform lifecycle was automated via a declarative `Makefile` and Python automation scripts:

- `make up`: Initializes the k3d cluster, provisions SeaweedFS, creates S3 buckets, downloads release weights, builds Docker images, and deploys the Ingress and HPA in one command.
- `make test`: Executes the Pytest unit test suite in `tests/test_unit.py`.
- `make down`: Completely tears down cluster resources and cleans up temporary local mounts.

Our GitHub Actions workflow (`.github/workflows/ci.yaml`) runs on every push and pull request across Ubuntu runners, implementing a strict five-stage verification pipeline.

6.2 Design Decisions and Engineering Rationale

- **Shift-Left Secret Scanning:** We integrated `gitleaks` as Stage 0 in CI to scan commit history for leaked credentials, ensuring that AWS keys or tokens are never committed.
- **Release-Coupled Evaluation:** Instead of rebuilding or training models inside CI (which would exceed GitHub Actions runner timeouts), Stage 3 downloads `model.tar.gz` directly from GitHub Release `v1.0.0`, verifies its SHA-256 hash, and scores predictions on `data/validation.jsonl`.
- **Automated Quality Gate:** Stage 4 enforces `score.py --min-validity 95`. If JSON validity falls below 95%, the pipeline fails with a non-zero exit code, blocking deployment.

6.3 Measured Empirical Results

- **Local Test Suite:** 4/4 tests passed in 0.12s covering cleaning rules, NACL simulation, schema validation, and VLSM math.
- **Static Analysis:** Ruff linting passed with **0 errors and 0 warnings**.
- **Secret Scanning:** Gitleaks scanned 100% of commits with **0 leaks detected**.
- **CI Execution Proofs (GitHub Actions):**
  - **Successful Pipeline Run:** https://github.com/KANAND6031/Cloud-Computing-Mini-Project/actions/runs/42502778393 completed in **2m 46s** with **98.0% JSON validity** (passing gate).
  - **Deliberately Failing Run:** https://github.com/KANAND6031/Cloud-Computing-Mini-Project/actions/runs/42458969182 tested against an untrained Pythia baseline (**0.0% JSON validity**), terminating with exit code 1 at Stage 4.

6.4 Key Assumptions and Modeling Boundaries

- *Assumption 1 (Docker Daemon Availability):* `make up` assumes Docker daemon is active locally with at least 4 GB available system RAM.
- *Assumption 2 (Public Release Access):* CI assumes GitHub Release assets are fetchable over the public internet via the GitHub REST API.

6.5 Identified Limitations and Proposed Improvements

- **Limitation (Lack of Semantic Gate):** The quality gate only enforces structural JSON validity ($\geq 95\%$), not semantic intent accuracy ($\geq 50\%$).
- **Improvement:** Add composite gating enforcing both $\geq 95\%$ JSON validity and $\geq 50\%$ intent accuracy.

7 Task 7: Performance Benchmarking, Cost Analysis, and Review

7.1 Architecture and Setup

We executed a comprehensive performance and stress evaluation against the live Kubernetes cluster using Locust with 4 concurrent users over a 2-minute test window (`tools/locustfile.py`), routing requests through the Traefik Ingress on port 8080. CPU and memory metrics were logged via `kubectl top pods`.

7.2 Measured Empirical Results

The benchmark results are summarized in Table 4.

Metric	Measurement Source / Tool	Measured Empirical Value
Total Requests Served	Locust Benchmark Summary	78 requests
System Throughput	Locust Benchmark Summary	**0.66 req/s** (2,393.16 req/hr)
Failed Requests	Locust HTTP 200 Verification	1 failure (initial client spawn timeout)
Measured Error Rate	Locust Error Calculation	**1.28%** (0.00% during steady-state)
Median Latency (p50)	Locust Summary Statistics	**5,600 ms** (steady-state p50 = 4,000 ms)
Tail Latency (p95)	Locust Summary Statistics	**17,000 ms** (steady-state p95 = 8,400 ms)
Average Latency	Locust Summary Statistics	5,903 ms (Min: 1,376 ms)
Peak Pod 1 Resource	`kubectl top pods`	996m CPU (99.6%) | **375.0 MiB RAM**
Peak Pod 2 Resource	`kubectl top pods`	1,004m CPU (100.4%) | **375.0 MiB RAM**
Median Cold Start	Task 4 Kubernetes Lifecycle Probing	**7.13 seconds**

Table 4: Task 7 comprehensive performance and resource utilization benchmark results.

7.3 Cost Model and Cloud Economic Breakdown

Per the provided pricing schedule in `data/rate_card.json`:

- CPU Core Allocation Rate: **INR 3.50 per core/hour**
- Memory Allocation Rate: **INR 0.45 per GiB/hour**

Each pod replica is provisioned with hard limits of 1 CPU core and 1 GiB RAM. The hourly operational cost per replica is:

$$\text{Cost}_{\text{replica}} = (1.0 \times 3.50) + (1.0 \times 0.45) = \text{INR } 3.95 \text{ / hour} \tag{1}$$

For our two active replicas, the total cluster compute cost is $2 \times 3.95 =$ **INR 7.90 / hour**.

Based on our empirical Locust throughput of 0.6647 req/s (2,393.16 req/hr), the cost to serve 1,000 user requests is:

$$\text{Cost per 1,000 requests} = \frac{\text{Hourly Cost of All Replicas}}{\text{Requests Served per Hour}} \times 1000 = \frac{\text{INR } 7.90}{2,393.16} \times 1000 = \text{INR } 3.301 \text{ INR} \tag{2}$$

Steady-State Projection: During the Task 4 steady-state benchmark (0.9056 req/s or 3,260.16 req/hr with 0% errors), the cost drops to INR 2.423 INR per 1,000 requests.

7.4 SLO Analysis and Error Budget Accounting

- **Service Level Objective (SLO):** "At least 99.0% of all valid HTTP `POST /parse` requests must return `HTTP 200 OK` with valid schema JSON within a 2-minute load-test window."
- **Allowed Error Budget:** Total Requests $\times (1 - 0.99) = 78 \times 0.01 =$ **0.78** requests.
- **Measured Budget Consumption:** During the 2-minute run, exactly 1 request failed due to connection timeout during initial worker thread saturation.

$$\text{Budget Consumed} = \frac{\text{Actual Failures}}{\text{Allowed Budget}} = \frac{1}{0.78} = \mathbf{128.2}\% \quad \text{(SLO breached by 0.22 requests)} \tag{3}$$

Comparison with Task 4 Rolling Update: In the Task 4 rolling update (104 requests, allowed budget = 1.04 requests), 0 failures occurred (0.0% budget consumed), demonstrating full SLO satisfaction during planned updates.

7.5 Component-by-Component Review and Architectural Roadmap

Table 5 presents our empirical architectural review across all 6 system dimensions.

Component	Measured Evidence	Identified Bottleneck / Limitation	Proposed Engineering Improvement
Data & Storage	2,761 cleaned records retained (92.0%); 37 hallucinated slots rejected	Static modulo sharding leads to uneven worker completion times	Implement dynamic pull-based message broker (RabbitMQ) for shard assignment
Model Quality	98.0% JSON validity achieved; full-frame accuracy is 11.0%	70M model capacity struggles with multi-entity slot extraction	Fine-tune a 0.5B model (Qwen2.5-0.5B) using LoRA adapters to boost full-frame accuracy
Latency & Throughput	Median latency p50 = 5,600 ms; p95 = 17,000 ms; 0.66 req/s	Autoregressive CPU token generation blocks Uvicorn asyncio event loop	Export model to ONNX Runtime with INT8 quantization; utilize Triton Inference Server
Resource Utilization	Pods consumed 1004m CPU (saturated); RAM hovered at 375 MiB (36.6%)	CPU is heavily bottlenecked while 63.4% of allocated memory sits unutilized	Rebalance pod resource limits to 2.0 CPU cores and 512 MiB memory for higher density
Reliability & Recovery	0.00% dropped requests during rolling update; recovery in 17.8s	Single-node k3d host lacks physical failure isolation	Deploy across multi-zone managed Kubernetes with topology spread constraints
Security & Cost	INR 3.301 per 1,000 reqs; 0 gitleaks detected; stateless NACL passed	Fixed pod allocations incur costs during idle traffic periods	Transition to Knative serverless scale-to-zero autoscaling on spot instances

Table 5: Task 7 comprehensive architectural review, empirical measurements, and roadmap.

7.6 Student Reflection and Project Takeaways

Working through Tasks 1 through 7 highlighted several key insights about building and running distributed machine learning workloads:

1. **Decoupling Storage from Compute:** Decoupling the model checkpoint from the container image and storing it in SeaweedFS S3 was essential. It kept the Docker image down to 351 MB of actual code and libraries, while allowing us to update from model v1 to v2 without rebuilding the base image or redeploying the container registry.
2. **Probes and Graceful Shutdown are Non-Negotiable:** A service that runs fine in isolation will fail under Kubernetes orchestration if probes and shutdown hooks are misconfigured. Without `/readyz`, traffic hits the container before weights are loaded; without `preStop: sleep 5`, Traefik sends requests to terminating pods during rolling updates.
3. **The Gap Between Theory and Cloud Reality:** Sequence-level distillation is conceptually straightforward, but CPU inference latency (1.4–2.9s per request) fundamentally limits single-pod throughput. Bridging this gap requires specialized serving runtimes (ONNX/Triton) and horizontal autoscaling.

Report compiled from LaTeX source (`report/report.tex`) adhering strictly to the 8-page assignment limit.